

✓ ByT5 Fine-tuning for Cuneiform Representation Comparison

Fine-tunes ByT5-small encoder + classification head on POS/NER tasks.

Run on Colab Pro+ with A100 GPU. Estimated: 12-16 hours for full sweep.

```
from google.colab import drive
drive.mount('/content/drive')
```

```
BASE_PATH = '/content/drive/MyDrive/FactGrid Cuneiform (AWCA)/people/Thompson/'
```

```
Mounted at /content/drive
```

```
!du -sh "/content/drive/MyDrive/FactGrid Cuneiform (AWCA)/people/Thompson/"/*
```

```
405M /content/drive/MyDrive/FactGrid Cuneiform (AWCA)/people/Thompson/alltexts_AKK.csv
16M /content/drive/MyDrive/FactGrid Cuneiform (AWCA)/people/Thompson/alltexts_SUX.csv
252K /content/drive/MyDrive/FactGrid Cuneiform (AWCA)/people/Thompson/BARTModelTrainingRound1&2(OUTDATED).ipynb
172K /content/drive/MyDrive/FactGrid Cuneiform (AWCA)/people/Thompson/BART Training Round 1.ipynb
130K /content/drive/MyDrive/FactGrid Cuneiform (AWCA)/people/Thompson/BART Training Round 2.ipynb
16K /content/drive/MyDrive/FactGrid Cuneiform (AWCA)/people/Thompson/Basello-Pedron-MEGA-table (!).docx
17K /content/drive/MyDrive/FactGrid Cuneiform (AWCA)/people/Thompson/Basello-Pedron-MEGA-table.docx
141K /content/drive/MyDrive/FactGrid Cuneiform (AWCA)/people/Thompson/BLEU_CHRF_result.ipynb
2.0M /content/drive/MyDrive/FactGrid Cuneiform (AWCA)/people/Thompson/Byt5 Cuneiform
3.4M /content/drive/MyDrive/FactGrid Cuneiform (AWCA)/people/Thompson/Byt5_Cuneiform_safe_autosave.ipynb
34K /content/drive/MyDrive/FactGrid Cuneiform (AWCA)/people/Thompson/byt5_cuneiform_withsanitycheck.ipynb
4.0K /content/drive/MyDrive/FactGrid Cuneiform (AWCA)/people/Thompson/byt5_final_models
4.0K /content/drive/MyDrive/FactGrid Cuneiform (AWCA)/people/Thompson/byt5_logs
54K /content/drive/MyDrive/FactGrid Cuneiform (AWCA)/people/Thompson/byt5_results
4.0K /content/drive/MyDrive/FactGrid Cuneiform (AWCA)/people/Thompson/byt5_runs
315K /content/drive/MyDrive/FactGrid Cuneiform (AWCA)/people/Thompson/cdli2cuneiform_2026.ipynb
297K /content/drive/MyDrive/FactGrid Cuneiform (AWCA)/people/Thompson/cdlinasu.ipynb
512 /content/drive/MyDrive/FactGrid Cuneiform (AWCA)/people/Thompson/Copy of UntN-Nasu texts Word-level.gsheet
82K /content/drive/MyDrive/FactGrid Cuneiform (AWCA)/people/Thompson/ElamiteExp4_Lemmatization.ipynb
332K /content/drive/MyDrive/FactGrid Cuneiform (AWCA)/people/Thompson/ElamiteExp5_LLM_Evaluation.ipynb
515K /content/drive/MyDrive/FactGrid Cuneiform (AWCA)/people/Thompson/ElamiteExperiments.ipynb
492K /content/drive/MyDrive/FactGrid Cuneiform (AWCA)/people/Thompson/ElamiteExperiments_v2.ipynb
551K /content/drive/MyDrive/FactGrid Cuneiform (AWCA)/people/Thompson/ElamiteExperiments_v3.ipynb
2.8M /content/drive/MyDrive/FactGrid Cuneiform (AWCA)/people/Thompson/Elamite_Lemma-base-draft.xlsx
1.5M /content/drive/MyDrive/FactGrid Cuneiform (AWCA)/people/Thompson/Elamite-Lemma-Base-Unicode.csv
512 /content/drive/MyDrive/FactGrid Cuneiform (AWCA)/people/Thompson/Elamite-Lemma-Base-Unicode.gsheet
403K /content/drive/MyDrive/FactGrid Cuneiform (AWCA)/people/Thompson/ElamiteNewData.ipynb
384K /content/drive/MyDrive/FactGrid Cuneiform (AWCA)/people/Thompson/ElamiteNewData_Unannotated.ipynb
43K /content/drive/MyDrive/FactGrid Cuneiform (AWCA)/people/Thompson/Eva_Glossed bisotun v1.xlsx
108K /content/drive/MyDrive/FactGrid Cuneiform (AWCA)/people/Thompson/Eva_Ideas on the theoretical background o
40K /content/drive/MyDrive/FactGrid Cuneiform (AWCA)/people/Thompson/Eva_Typology cheat sheet - general.docx
47K /content/drive/MyDrive/FactGrid Cuneiform (AWCA)/people/Thompson/GraphAnalysis_ThreeLanguages.ipynb
4.0K /content/drive/MyDrive/FactGrid Cuneiform (AWCA)/people/Thompson/graph_outputs
512 /content/drive/MyDrive/FactGrid Cuneiform (AWCA)/people/Thompson/Lab Notes Thompson.gdoc
170K /content/drive/MyDrive/FactGrid Cuneiform (AWCA)/people/Thompson/Nasu_CDLI_strings.csv
512 /content/drive/MyDrive/FactGrid Cuneiform (AWCA)/people/Thompson/Nasu_CDLI_strings.gsheet
13M /content/drive/MyDrive/FactGrid Cuneiform (AWCA)/people/Thompson/oip092hallock1969glossary.pdf
296K /content/drive/MyDrive/FactGrid Cuneiform (AWCA)/people/Thompson/Probability_stats
4.0K /content/drive/MyDrive/FactGrid Cuneiform (AWCA)/people/Thompson/Split_trainings
4.5K /content/drive/MyDrive/FactGrid Cuneiform (AWCA)/people/Thompson/three_lang_results.json
81K /content/drive/MyDrive/FactGrid Cuneiform (AWCA)/people/Thompson/ThreeLanguagePipeline_Full.ipynb
195K /content/drive/MyDrive/FactGrid Cuneiform (AWCA)/people/Thompson/ThreeLanguagePipeline_v2.ipynb
235K /content/drive/MyDrive/FactGrid Cuneiform (AWCA)/people/Thompson/to_unicode_nasu_clean.csv
512 /content/drive/MyDrive/FactGrid Cuneiform (AWCA)/people/Thompson/to_unicode_nasu_clean.gsheet
232K /content/drive/MyDrive/FactGrid Cuneiform (AWCA)/people/Thompson/to_unicode_nasu_clean_new.csv
512 /content/drive/MyDrive/FactGrid Cuneiform (AWCA)/people/Thompson/to_unicode_nasu_clean_new.gsheet
19K /content/drive/MyDrive/FactGrid Cuneiform (AWCA)/people/Thompson/unmatched.csv
512 /content/drive/MyDrive/FactGrid Cuneiform (AWCA)/people/Thompson/unmatchednew_AAedit.gsheet
2.0K /content/drive/MyDrive/FactGrid Cuneiform (AWCA)/people/Thompson/unmatchednew_AAedit - unmatchednew.csv
461K /content/drive/MyDrive/FactGrid Cuneiform (AWCA)/people/Thompson/unmatchednew.csv
6.0K /content/drive/MyDrive/FactGrid Cuneiform (AWCA)/people/Thompson/unmatchednewfinal.csv
512 /content/drive/MyDrive/FactGrid Cuneiform (AWCA)/people/Thompson/unmatchednewfinal.gsheet
512 /content/drive/MyDrive/FactGrid Cuneiform (AWCA)/people/Thompson/unmatchednew.gsheet
512 /content/drive/MyDrive/FactGrid Cuneiform (AWCA)/people/Thompson/unmatchednew - solo.csv
512 /content/drive/MyDrive/FactGrid Cuneiform (AWCA)/people/Thompson/unmatchednew - solo.gsheet
512 /content/drive/MyDrive/FactGrid Cuneiform (AWCA)/people/Thompson/unmatchednew - solonew.csv
137K /content/drive/MyDrive/FactGrid Cuneiform (AWCA)/people/Thompson/UntN-Nasu texts Word-level.csv
512 /content/drive/MyDrive/FactGrid Cuneiform (AWCA)/people/Thompson/UntN-Nasu texts Word-level.gsheet
```

✓ Cell 1: Install

```
!pip install -q transformers sentencepiece protobuf
!pip install -q scikit-learn pandas openpyxl regex
```

Cell 2: Imports

```
import pandas as pd
import numpy as np
import regex as re
import json, os, gc, time
from collections import Counter
from sklearn.model_selection import StratifiedKFold
from sklearn.metrics import f1_score
from sklearn.preprocessing import LabelEncoder
import torch
import torch.nn as nn
from torch.utils.data import Dataset, DataLoader
from transformers import AutoTokenizer, AutoModel
import warnings
warnings.filterwarnings('ignore')

SEED = 42
MIN_CLASS_COUNT = 20
np.random.seed(SEED)
torch.manual_seed(SEED)

device = torch.device('cuda' if torch.cuda.is_available() else 'cpu')
print(f"Device: {device}")
if torch.cuda.is_available():
    print(f"GPU: {torch.cuda.get_device_name(0)}")
    mem = torch.cuda.get_device_properties(0).total_memory / 1e9
    print(f"Memory: {mem:.1f} GB")
    print(f"bf16 supported: {torch.cuda.is_bf16_supported()}")
```

```
Device: cuda
GPU: NVIDIA A100-SXM4-40GB
Memory: 42.4 GB
bf16 supported: True
```

Cell 3: POS Harmonization Maps

```
POS_UNIFIED_MAP = {
    'akk': {
        'N': 'NOUN', 'V': 'VERB', 'AJ': 'ADJ', 'AV': 'ADV',
        'PRP': 'ADP', 'DET': 'DET', 'CNJ': 'CONJ', 'MOD': 'MOD',
        'REL': 'REL', 'SBJ': 'SBJN', 'IP': 'INTJ',
        'PN': 'PN', 'DN': 'DN', 'GN': 'GN', 'CN': 'CN',
        'RN': 'RN', 'QN': 'QN', 'WN': 'WN', 'MN': 'MN',
        'AN': 'AN', 'FN': 'FN', 'TN': 'TN', 'LN': 'LN', 'ON': 'ON',
        'n': 'NUM', 'u': 'X', 'X': 'X',
    },
    'sux': {
        'N': 'NOUN', 'V/t': 'VERB', 'V/i': 'VERB', 'V': 'VERB',
        'AJ': 'ADJ', 'AV': 'ADV',
        'NU': 'NUM', 'IP': 'INTJ', 'QP': 'QP',
        'DN': 'DN', 'GN': 'GN', 'PN': 'PN', 'RN': 'RN',
        'SN': 'SN', 'TN': 'TN', 'WN': 'WN', 'MN': 'MN',
        'NA': 'X',
    },
    'elx': {
        'Noun': 'NOUN', 'Verb': 'VERB', 'ADJ': 'ADJ', 'other': 'OTHER',
        'PN': 'PN', 'PN-hyp': 'PN', 'GN': 'GN', 'DN': 'DN', 'Magic': 'OTHER',
    },
}

ENTITY_TAGS = {'PN', 'DN', 'GN', 'CN', 'RN', 'QN', 'WN', 'MN', 'AN', 'FN', 'TN', 'LN', 'ON', 'SN', 'QP'}

def map_pos_unified(pos_raw, lang):
    return POS_UNIFIED_MAP.get(lang, {}).get(pos_raw, 'X')

def map_pos_grammatical(pos_unified):
    return 'PROPN' if pos_unified in ENTITY_TAGS else pos_unified
```

```
def map_ner_tag(pos_unified):
    return pos_unified if pos_unified in ENTITY_TAGS else 'O'

print("POS maps loaded.")
```

POS maps loaded.

- Cell 4: Sign Lists

```
sign_list = pd.read_json(
    'https://raw.githubusercontent.com/situx/Nuolenna/master/sign_list.json', orient='index')
sign_list.columns = ['unicode']
sign_list['sign'] = sign_list.index.tolist()
sign_list = sign_list[['sign', 'unicode']].reset_index(drop=True)

akkademia = pd.read_csv(
    'https://raw.githubusercontent.com/gaiguthertz/Akkademia/master/cuneiform_to_unicode_fixed.csv')
merged = pd.merge(sign_list, akkademia, on=['sign', 'unicode'], how='outer')
sign_dict = dict(zip(merged['sign'].astype(str), merged['unicode'].astype(str)))

UNMATCHED_PATH_1 = BASE_PATH + 'unmatchednew_AAedit - unmatchednew.csv'
UNMATCHED_PATH_2 = BASE_PATH + 'unmatchednew - solonew.csv'
manual_dict, manual_dict2 = {}, {}

try:
    unmatched = pd.read_csv(UNMATCHED_PATH_1)[['unmatched_sign', 'use']].dropna()
    unmatched2 = pd.read_csv(UNMATCHED_PATH_2)[['value', 'SIGN']].dropna()
    manual_dict = dict(zip(unmatched['unmatched_sign'], unmatched['use']))
    manual_dict2 = dict(zip(unmatched2['value'].str.strip("[]' "), unmatched2['SIGN']))
    sign_dict.update(manual_dict)
    sign_dict.update(manual_dict2)
    print(f"Manual corrections: {len(manual_dict)} + {len(manual_dict2)}")
except FileNotFoundError:
    print("No manual corrections found.")

print(f"Total sign mappings: {len(sign_dict)}")

Manual corrections: 114 + 22
Total sign mappings: 18271
```

- Cell 5: Unicode Conversion Functions

```
def normalize_transliteration(text, lang='akk'):
    if pd.isna(text) or text == '':
        return ''
    s = str(text)
    s = re.sub(r'[\^]*\\}', '', s)
    s = s.replace('-', ' ').replace('.', ' ')
    for ch in ['[', ']', '#', '!', '?', '*', '(', ')']:
        s = s.replace(ch, ' ')
    return re.sub(r'\s+', ' ', s).strip()

def transliteration_to_unicode(text, sign_dict, lang='akk'):
    normalized = normalize_transliteration(text, lang)
    if not normalized:
        return '', []
    tokens = normalized.split()
    out, unmatched = [], []
    for tok in tokens:
        if tok in sign_dict:
            out.append(sign_dict[tok])
        elif tok.lower() in sign_dict:
            out.append(sign_dict[tok.lower()])
        elif tok.upper() in sign_dict:
            out.append(sign_dict[tok.upper()])
        else:
            out.append(tok)
            if re.search(r'\p{Latin}', tok):
                unmatched.append(tok)
    return ' '.join(out), unmatched

def convert_column_to_unicode(forms, sign_dict, lang='akk'):
    results = forms.apply(lambda x: transliteration_to_unicode(x, sign_dict, lang))
    unicode_col = results.apply(lambda x: x[0])
```

```
n_clean = (results.apply(lambda x: len(x[1]) == 0)).sum()
return unicode_col, n_clean / len(forms)
```

```
print("Unicode functions defined.")
```

```
Unicode functions defined.
```

Cell 6: Data Loaders

```
def load_akkadian(csv_path):
    df = pd.read_csv(csv_path, low_memory=False)
    df = df[df['pos'].notna() & (df['pos'] != 'u')].copy()
    df = df[df['form'].notna() & (df['form'] != 'x') & (df['form'] != 'X')].copy()
    out = pd.DataFrame({
        'token_id': range(len(df)),
        'text_id': df['id_text'].values,
        'form_latin': df['form'].values,
        'pos_raw': df['pos'].values,
        'lemma': df['cf'].values,
        'language': 'akk',
    })
    out['pos_unified'] = out['pos_raw'].apply(lambda x: map_pos_unified(x, 'akk'))
    out['pos_grammatical'] = out['pos_unified'].apply(map_pos_grammatical)
    return out

def load_sumerian(csv_path):
    df = pd.read_csv(csv_path)
    df = df[df['pos'].notna() & (df['pos'] != '') & (df['pos'].astype(str) != 'nan')].copy()
    df = df[df['form'].notna() & (df['form'] != 'x') & (df['form'] != 'X')].copy()
    out = pd.DataFrame({
        'token_id': range(len(df)),
        'text_id': df['id_text'].values,
        'form_latin': df['form'].values,
        'pos_raw': df['pos'].values,
        'lemma': df['cf'].values,
        'language': 'sux',
    })
    out['pos_unified'] = out['pos_raw'].apply(lambda x: map_pos_unified(x, 'sux'))
    out['pos_grammatical'] = out['pos_unified'].apply(map_pos_grammatical)
    return out

def add_unicode(df, sign_dict):
    df['form_unicode'], rate = convert_column_to_unicode(
        df['form_latin'], sign_dict, df['language'].iloc[0])
    print(f" Unicode rate: {rate:.1%}")
    return df

print("Loaders defined.")
```

```
Loaders defined.
```

Cell 7: Load All Data

```
print("Loading Akkadian...")
akk = add_unicode(load_akkadian(BASE_PATH + 'alltexts_AKK.csv'), sign_dict)
print(f" {len(akk):,} tokens")

print("\nLoading Sumerian...")
sux = add_unicode(load_sumerian(BASE_PATH + 'alltexts_SUX.csv'), sign_dict)
print(f" {len(sux):,} tokens")

print("\nLoading Elamite...")
DICT_PATH = BASE_PATH + 'Elamite_Lemma-base-draft.xlsx'
all_sheets = pd.read_excel(DICT_PATH, sheet_name=None)
target_tabs = ['ADJ', 'Noun', 'Verb', 'other', 'PN', 'PN-hyp', 'GN', 'DN', 'Magic']
columns_to_keep = ['transliteration', 'sorting', 'period', 'base', 'logogram',
    'morpheme_1', 'morpheme_2', 'morpheme_3', 'sense_hk', 'certainty-weight_hk',
    'sense_hk_qid', 'certainty-weight_MEGA', 'sense_MEGA_qid', 'POS', 'number', 'person']
combined = []
for name in target_tabs:
    if name in all_sheets:
        df = all_sheets[name]
        df['category'] = name
```

```

existing = [c for c in columns_to_keep if c in df.columns]
combined.append(df[existing + ['category']])
final_dict = pd.concat(combined, ignore_index=True)

to_uni = final_dict.copy()
to_uni['transliteration'] = to_uni['transliteration'].astype(str).fillna('')
to_uni['transliteration'] = to_uni['transliteration'].str.replace('-', ' ')
for ch in ['.', '[', ']', '*', '!', '?', '/', ' ', ':', ';', '^', ``]:
    to_uni['transliteration'] = to_uni['transliteration'].str.replace(ch, '', regex=False)
to_uni['transliteration'] = to_uni['transliteration'].str.replace('.', ' ', regex=False)
to_uni['transliteration'] = to_uni['transliteration'].str.lower()
to_uni['transliteration'] = to_uni['transliteration'].str.replace('X', '', regex=False)

def replace_unicode(text):
    return ' '.join([sign_dict.get(s, s) for s in text.split()])

to_uni['unicode'] = to_uni['transliteration'].apply(replace_unicode)
to_uni['roman'] = to_uni['unicode'].apply(
    lambda x: any(bool(re.search(r'\p{Latin}', w)) for w in x.split()))

# Second pass
unicode_dict_v2 = dict(zip(merged['sign'].astype(str), merged['unicode'].astype(str)))
unicode_dict_v2.update(manual_dict)
unicode_dict_v2.update(manual_dict2)

def norm_v2(s):
    if pd.isna(s): return ""
    s = str(s)
    s = re.sub(r"\\(\s*md\s*)", " m d ", s)
    s = re.sub(r"[.,;!?()\[\]\{\}<>\\\"'/*^~]", " ", s)
    s = re.sub(r"[-\u2013\u2014]", " ", s)
    s = re.sub(r"[\u2080-\u2089]+", "", s)
    s = re.sub(r"\d+", "", s)
    return re.sub(r"\s+", " ", s.lower().strip())

still = to_uni[to_uni['roman']].copy()
still['unicode_v2'] = still['transliteration'].apply(
    lambda x: ' '.join([unicode_dict_v2.get(t, t) for t in norm_v2(x).split()]))
still['roman_v2'] = still['unicode_v2'].apply(
    lambda x: any(bool(re.search(r'\p{Latin}', w)) for w in x.split()))
to_uni.loc[still.index, 'unicode'] = still['unicode_v2']
to_uni.loc[still.index, 'roman'] = still['roman_v2']

elx_clean = to_uni[~to_uni['roman']].copy()
elx = pd.DataFrame({
    'token_id': range(len(elx_clean)),
    'text_id': 'dict_' + elx_clean.index.astype(str),
    'form_latin': elx_clean['transliteration'].values,
    'form_unicode': elx_clean['unicode'].values,
    'pos_raw': elx_clean['category'].values,
    'language': 'elx',
})
elx['pos_unified'] = elx['pos_raw'].apply(lambda x: map_pos_unified(x, 'elx'))
elx['pos_grammatical'] = elx['pos_unified'].apply(map_pos_grammatical)
print(f" {len(elx):,} clean entries")

datasets = {'akk': akk, 'sux': sux, 'elx': elx}

for lang, df in datasets.items():
    df['entity_detect'] = df['pos_unified'].apply(
        lambda x: 'ENTITY' if x in ENTITY_TAGS else 'NON-ENTITY')
    df['entity_type'] = df['pos_unified'].apply(
        lambda x: x if x in ENTITY_TAGS else 'NON-ENTITY')

print("\nAll data loaded:")
for lang, df in datasets.items():
    print(f" {lang.upper():,}: {len(df):,} tokens")

Loading Akkadian...
Unicode rate: 98.9%
1,255,669 tokens

Loading Sumerian...
Unicode rate: 99.9%
146,143 tokens

Loading Elamite...
13,322 clean entries

```

```
All data loaded:
AKK: 1,255,669 tokens
SUX: 146,143 tokens
ELX: 13,322 tokens
```

Cell 8: ByT5 Classifier Model

```
class ByT5Classifier(nn.Module):
    """ByT5 encoder + mean pooling + classification head."""
    def __init__(self, n_classes, dropout=0.3):
        super().__init__()
        self.encoder = AutoModel.from_pretrained('google/byt5-small').encoder
        self.hidden_size = self.encoder.config.d_model
        self.dropout = nn.Dropout(dropout)
        self.classifier = nn.Linear(self.hidden_size, n_classes)
        print(f"    ByT5 encoder hidden size: {self.hidden_size}")

    def forward(self, input_ids, attention_mask):
        out = self.encoder(input_ids=input_ids, attention_mask=attention_mask)
        mask = attention_mask.unsqueeze(-1).float()
        pooled = (out.last_hidden_state * mask).sum(1) / mask.sum(1).clamp(min=1)
        return self.classifier(self.dropout(pooled))

print("Model class defined.")

Model class defined.
```

Cell 9: Tokenizer + Sanity Check

```
print("Loading tokenizer...")
tokenizer = AutoTokenizer.from_pretrained('google/byt5-small')

def tokenize_batch(texts, max_len=128):
    enc = tokenizer(texts, max_length=max_len, padding='max_length',
                    truncation=True, return_tensors='pt')
    return enc['input_ids'], enc['attention_mask']

# Sanity check tokenizer
test_ids, test_mask = tokenize_batch(["hello", "𐌆𐌵𐌹𐌺𐌰"])
print(f"Tokenizer check:")
print(f" 'hello' tokens: {test_ids[0][:10].tolist()}")
print(f" '𐌆𐌵𐌹𐌺𐌰' tokens: {test_ids[1][:15].tolist()}")
print(f" Shapes: {test_ids.shape}")

Loading tokenizer...
config.json: 100% 698/698 [00:00<00:00, 91.2kB/s]

tokenizer_config.json: 2.59k/? [00:00<00:00, 349kB/s]

special_tokens_map.json: 2.50k/? [00:00<00:00, 335kB/s]

Warning: You are sending unauthenticated requests to the HF Hub. Please set a HF_TOKEN to enable higher rate limits a
WARNING:huggingface_hub.utils._http:Warning: You are sending unauthenticated requests to the HF Hub. Please set a HF_
Tokenizer check:
'hello' tokens: [107, 104, 111, 111, 114, 1, 0, 0, 0, 0]
'𐌆𐌵𐌹𐌺𐌰' tokens: [243, 149, 131, 131, 243, 149, 139, 193, 1, 0, 0, 0, 0, 0, 0]
Shapes: torch.Size([2, 128])
```

Cell 10: Training Function (Manual PyTorch Loop)

```
def run_byt5_experiment(texts, labels_str, name="", max_len=128,
                        n_epochs=15, batch_size=16, lr=5e-4):
    """Train ByT5 encoder + classifier with 3-fold CV. Returns mean F1."""
    le = LabelEncoder()
    labels = le.fit_transform(labels_str)
    n_classes = len(le.classes_)

    print(f"    Tokenizing {len(texts)} texts (max_len={max_len})...")
    all_ids, all_mask = tokenize_batch(texts, max_len)
    all_labels = torch.LongTensor(labels)
```

```

cv = StratifiedKFold(n_splits=3, shuffle=True, random_state=SEED)
fold_f1s = []

for fold, (train_idx, val_idx) in enumerate(cv.split(texts, labels)):
    t0 = time.time()
    print(f"        Fold {fold+1}/3: ", end="", flush=True)

    tr_ids = all_ids[train_idx].to(device)
    tr_mask = all_mask[train_idx].to(device)
    tr_labels = all_labels[train_idx].to(device)
    va_ids = all_ids[val_idx].to(device)
    va_mask = all_mask[val_idx].to(device)
    va_labels = all_labels[val_idx].numpy()

    model = ByT5Classifier(n_classes).to(device)
    optimizer = torch.optim.AdamW(model.parameters(), lr=lr, weight_decay=0.01)
    scheduler = torch.optim.lr_scheduler.CosineAnnealingLR(optimizer, T_max=n_epochs)

    class_counts = np.bincount(labels[train_idx], minlength=n_classes).astype(float)
    class_counts = np.maximum(class_counts, 1.0)
    weights = torch.FloatTensor(1.0 / class_counts).to(device)
    weights = weights / weights.sum() * n_classes
    criterion = nn.CrossEntropyLoss(weight=weights)

    best_val_loss = float('inf')
    best_state = None
    patience_counter = 0
    n_train = len(train_idx)
    use_bf16 = torch.cuda.is_bf16_supported()

    for epoch in range(n_epochs):
        model.train()
        perm = torch.randperm(n_train)
        epoch_loss = 0.0
        n_batches = 0

        for i in range(0, n_train, batch_size):
            idx = perm[i:i+batch_size]
            b_ids = tr_ids[idx]
            b_mask = tr_mask[idx]
            b_labels = tr_labels[idx]

            optimizer.zero_grad()

            if use_bf16:
                with torch.amp.autocast('cuda', dtype=torch.bfloat16):
                    logits = model(b_ids, b_mask)
                    loss = criterion(logits, b_labels)
            else:
                logits = model(b_ids, b_mask)
                loss = criterion(logits, b_labels)

            if torch.isnan(loss):
                print(f"NaN! ", end="")
                continue

            loss.backward()
            torch.nn.utils.clip_grad_norm_(model.parameters(), 1.0)
            optimizer.step()
            epoch_loss += loss.item()
            n_batches += 1

        scheduler.step()
        avg_loss = epoch_loss / max(n_batches, 1)

        # Validation
        model.eval()
        with torch.no_grad():
            val_logits_list = []
            for j in range(0, len(va_ids), batch_size * 2):
                if use_bf16:
                    with torch.amp.autocast('cuda', dtype=torch.bfloat16):
                        vl = model(va_ids[j:j+batch_size*2], va_mask[j:j+batch_size*2])
                        val_logits_list.append(vl.float())
                else:
                    vl = model(va_ids[j:j+batch_size*2], va_mask[j:j+batch_size*2])

```

```

        val_logits_list.append(vl)
    val_logits = torch.cat(val_logits_list, dim=0)
    val_loss = nn.CrossEntropyLoss()(
        val_logits, torch.LongTensor(va_labels).to(device)).item()

    if (epoch + 1) % 5 == 0 or epoch == 0:
        print(f"e{epoch+1}({avg_loss:.3f}/{val_loss:.3f}) ", end="", flush=True)

    if val_loss < best_val_loss:
        best_val_loss = val_loss
        best_state = {k: v.cpu().clone() for k, v in model.state_dict().items()}
        patience_counter = 0
    else:
        patience_counter += 1
        if patience_counter >= 4:
            print(f"[stop@{epoch+1}] ", end="", flush=True)
            break

    # Evaluate best model
    model.load_state_dict(best_state)
    model.eval()
    with torch.no_grad():
        all_preds = []
        for j in range(0, len(va_ids), batch_size * 2):
            logits = model(va_ids[j:j+batch_size*2], va_mask[j:j+batch_size*2])
            all_preds.append(logits.argmax(dim=1).cpu().numpy())
        preds = np.concatenate(all_preds)

    f1 = f1_score(va_labels, preds, average='macro', zero_division=0)
    elapsed = time.time() - t0
    print(f"F1={f1:.4f} ({elapsed:.0f}s)")
    fold_f1s.append(f1)

    del model, tr_ids, tr_mask, tr_labels, va_ids, va_mask
    gc.collect()
    torch.cuda.empty_cache()

    mean_f1 = np.mean(fold_f1s)
    std_f1 = np.std(fold_f1s)
    return mean_f1, std_f1

print("Training function defined.")

```

Training function defined.

✓ Cell 11: SANITY CHECK

Run this first! If F1 > 0, proceed to full experiments.

```

print("=" * 60)
print("  SANITY CHECK: tiny test on 500 AKK tokens")
print("=" * 60)

test_df = akk.sample(500, random_state=SEED)
counts = test_df['pos_unified'].value_counts()
valid = counts[counts >= 10].index.tolist()
test_df = test_df[test_df['pos_unified'].isin(valid)]
test_texts = test_df['form_latin'].astype(str).tolist()
test_labels = test_df['pos_unified'].astype(str).tolist()

print(f"  {len(test_texts)} tokens, {len(set(test_labels))} classes")

f1, std = run_byt5_experiment(
    test_texts, test_labels, name="sanity",
    max_len=64, n_epochs=10, batch_size=32, lr=5e-4
)
print(f"\n  Sanity check F1: {f1:.4f} (±{std:.4f})")

if f1 < 0.01:
    print("\n  *** MODEL NOT LEARNING - DEBUG BEFORE PROCEEDING ***")
    m = ByT5Classifier(5).to(device)
    trainable = sum(p.requires_grad for p in m.parameters())
    total = sum(1 for _ in m.parameters())
    print(f"  Trainable params: {trainable}/{total}")
    del m

```



```

else:
    print(f"\n Model is learning! Proceed to Cell 12.")

=====
SANITY CHECK: tiny test on 500 AKK tokens
=====
454 tokens, 11 classes
Tokenizing 454 texts (max_len=64)...
Fold 1/3:
pytorch_model.bin: 100%                               1.20G/1.20G [00:03<00:00, 679MB/s]

Loading weights: 100%                                171/171 [00:00<00:00, 1067.88it/s, Materializing param=shared.weight]
The tied weights mapping and config for this model specifies to tie shared.weight to encoder.embed_tokens.weight, but
model.safetensors: 100%                             1.20G/1.20G [00:03<00:00, 1.00GB/s]
The tied weights mapping and config for this model specifies to tie shared.weight to decoder.embed_tokens.weight, but
T5Model LOAD REPORT from: google/byt5-small
Key          | Status      | |
-----+-----+
lm_head.weight | UNEXPECTED | |

Notes:
- UNEXPECTED :can be ignored when loading from different task/architecture; not ok if you expect identical arch.
  ByT5 encoder hidden size: 1472
e1(2.403/2.387) e5(2.231/2.106) e10(1.916/1.995) F1=0.2371 (20s)
Fold 2/3:
Loading weights: 100%                                171/171 [00:00<00:00, 1255.18it/s, Materializing param=shared.weight]
The tied weights mapping and config for this model specifies to tie shared.weight to encoder.embed_tokens.weight, but
The tied weights mapping and config for this model specifies to tie shared.weight to decoder.embed_tokens.weight, but
T5Model LOAD REPORT from: google/byt5-small
Key          | Status      | |
-----+-----+
lm_head.weight | UNEXPECTED | |

Notes:
- UNEXPECTED :can be ignored when loading from different task/architecture; not ok if you expect identical arch.
  ByT5 encoder hidden size: 1472
e1(2.398/2.342) e5(2.289/2.168) e10(2.012/2.039) F1=0.1847 (12s)
Fold 3/3:
Loading weights: 100%                                171/171 [00:00<00:00, 1129.40it/s, Materializing param=shared.weight]
The tied weights mapping and config for this model specifies to tie shared.weight to encoder.embed_tokens.weight, but
The tied weights mapping and config for this model specifies to tie shared.weight to decoder.embed_tokens.weight, but
T5Model LOAD REPORT from: google/byt5-small
Key          | Status      | |
-----+-----+
lm_head.weight | UNEXPECTED | |

Notes:
- UNEXPECTED :can be ignored when loading from different task/architecture; not ok if you expect identical arch.
  ByT5 encoder hidden size: 1472
e1(2.397/2.359) e5(2.211/2.135) e10(1.898/1.878) F1=0.2818 (12s)

Sanity check F1: 0.2345 (±0.0397)

Model is learning! Proceed to Cell 12.

```

▼ Cell 12: Full Experiments

Only run after sanity check passes.

```

TASKS = [
    ('pos_unified', 'unified_pos'),
    ('pos_grammatical', 'gram_pos'),
    ('entity_detect', 'ent_detect'),
    ('entity_type', 'ent_type'),
]

MAX_SAMPLES = 5000
byt5_results = {}

for lang, df in datasets.items():
    print(f"\n{'=' * 60}")
    print(f"  ByT5 - {lang.upper()}")
    print(f"{'=' * 60}")
    byt5_results[lang] = {}

```

```

for pos_col, task_name in TASKS:
    if pos_col not in df.columns:
        print(f"\n Skip {task_name}: column missing")
        continue

    counts = df[pos_col].value_counts()
    valid = counts[counts >= MIN_CLASS_COUNT].index.tolist()
    task_df = df[df[pos_col].isin(valid)].copy()
    if len(task_df) > MAX_SAMPLES:
        task_df = task_df.sample(MAX_SAMPLES, random_state=SEED)
    if len(valid) < 2:
        print(f"\n Skip {task_name}: <2 classes")
        continue

    texts_l = task_df['form_latin'].astype(str).tolist()
    texts_u = task_df['form_unicode'].astype(str).tolist()
    texts_c = [f"{l} | {u}" for l, u in zip(texts_l, texts_u)]
    labels = task_df[pos_col].astype(str).tolist()

    print(f"\n {task_name} ({len(task_df):,} tokens, {len(valid)} classes)")

    print(f" Latin:")
    f1_l, std_l = run_byt5_experiment(
        texts_l, labels, f"{lang}_{task_name}_L",
        max_len=128, n_epochs=15, batch_size=16)
    print(f" -> {f1_l:.4f} (±{std_l:.4f})")

    print(f" Unicode:")
    f1_u, std_u = run_byt5_experiment(
        texts_u, labels, f"{lang}_{task_name}_U",
        max_len=128, n_epochs=15, batch_size=16)
    print(f" -> {f1_u:.4f} (±{std_u:.4f})")

    print(f" Concat:")
    f1_c, std_c = run_byt5_experiment(
        texts_c, labels, f"{lang}_{task_name}_C",
        max_len=256, n_epochs=15, batch_size=8)
    print(f" -> {f1_c:.4f} (±{std_c:.4f})")

    gain = f1_c - max(f1_l, f1_u)
    marker = '***' if gain > 0.01 else ('+' if gain > 0 else '-')
    print(f" Gain: {gain:+.4f} {marker}")

    byt5_results[lang][task_name] = {
        'latin': f1_l, 'unicode': f1_u, 'concat': f1_c, 'gain': gain}

    with open('/content/byt5_results.json', 'w') as f:
        json.dump(byt5_results, f, indent=2)
    print(f" [Checkpoint saved]")

```



```

=====
ByT5 - AKK
=====

unified_pos (5,000 tokens, 24 classes)
Latin:
  Tokenizing 5000 texts (max_len=128)...
  Fold 1/3:
Loading weights: 100%                                171/171 [00:00<00:00, 1176.46it/s, Materializing param=shared.weight]
The tied weights mapping and config for this model specifies to tie shared.weight to encoder.embed_tokens.weight, bu
The tied weights mapping and config for this model specifies to tie shared.weight to decoder.embed_tokens.weight, bu
T5Model LOAD REPORT from: google/byt5-small
Key          | Status      | |
-----+-----+
lm_head.weight | UNEXPECTED | |

Notes:
- UNEXPECTED :can be ignored when loading from different task/architecture; not ok if you expect identical arch.
  ByT5 encoder hidden size: 1472
e1(2.917/2.329) e5(1.366/1.355) e10(0.634/1.066) e15(0.386/0.996) F1=0.5636 (257s)
  Fold 2/3:
Loading weights: 100%                                171/171 [00:00<00:00, 1165.25it/s, Materializing param=shared.weight]
The tied weights mapping and config for this model specifies to tie shared.weight to encoder.embed_tokens.weight, bu
The tied weights mapping and config for this model specifies to tie shared.weight to decoder.embed_tokens.weight, bu
T5Model LOAD REPORT from: google/byt5-small
Key          | Status      | |
-----+-----+
lm_head.weight | UNEXPECTED | |

Notes:
- UNEXPECTED :can be ignored when loading from different task/architecture; not ok if you expect identical arch.
  ByT5 encoder hidden size: 1472
e1(2.806/2.220) e5(1.158/1.339) e10(0.551/1.047) [stop@14] F1=0.5678 (240s)
  Fold 3/3:
Loading weights: 100%                                171/171 [00:00<00:00, 1324.04it/s, Materializing param=shared.weight]
The tied weights mapping and config for this model specifies to tie shared.weight to encoder.embed_tokens.weight, bu
The tied weights mapping and config for this model specifies to tie shared.weight to decoder.embed_tokens.weight, bu
T5Model LOAD REPORT from: google/byt5-small
Key          | Status      | |
-----+-----+
lm_head.weight | UNEXPECTED | |

Notes:
- UNEXPECTED :can be ignored when loading from different task/architecture; not ok if you expect identical arch.
  ByT5 encoder hidden size: 1472
e1(3.070/2.676) e5(1.418/1.307) e10(0.600/1.040) e15(0.332/0.986) [stop@15] F1=0.5817 (256s)
-> 0.5710 (±0.0077)
Unicode:
  Tokenizing 5000 texts (max_len=128)...
  Fold 1/3:
Loading weights: 100%                                171/171 [00:00<00:00, 1185.04it/s, Materializing param=shared.weight]
The tied weights mapping and config for this model specifies to tie shared.weight to encoder.embed_tokens.weight, bu
The tied weights mapping and config for this model specifies to tie shared.weight to decoder.embed_tokens.weight, bu
T5Model LOAD REPORT from: google/byt5-small
Key          | Status      | |
-----+-----+
lm_head.weight | UNEXPECTED | |

Notes:
- UNEXPECTED :can be ignored when loading from different task/architecture; not ok if you expect identical arch.
  ByT5 encoder hidden size: 1472
e1(3.046/2.775) e5(2.085/2.142) e10(1.179/1.547) e15(0.760/1.475) F1=0.4025 (257s)
  Fold 2/3:
Loading weights: 100%                                171/171 [00:00<00:00, 1203.33it/s, Materializing param=shared.weight]
The tied weights mapping and config for this model specifies to tie shared.weight to encoder.embed_tokens.weight, bu
The tied weights mapping and config for this model specifies to tie shared.weight to decoder.embed_tokens.weight, bu
T5Model LOAD REPORT from: google/byt5-small
Key          | Status      | |
-----+-----+
lm_head.weight | UNEXPECTED | |

Notes:
- UNEXPECTED :can be ignored when loading from different task/architecture; not ok if you expect identical arch.
  ByT5 encoder hidden size: 1472
e1(2.984/2.490) e5(2.286/2.259) e10(1.466/1.718) e15(1.005/1.624) F1=0.3448 (257s)
  Fold 3/3:
Loading weights: 100%                                171/171 [00:00<00:00, 1226.27it/s, Materializing param=shared.weight]
The tied weights mapping and config for this model specifies to tie shared.weight to encoder.embed_tokens.weight, bu

```